ETHEREUM PROOF OF STAKE ALGORITHM

Stake, not Compute

- In Proof-of-Work, miners compete by burning electricity to solve hash puzzles.
- The winner proposes the next block. I
- n Proof-of-Stake, validators compete by *locking up* (staking) 32 ETH as collateral.
- Selection is pseudo-random, weighted by stake. The key insight: instead of wasting energy to make cheating expensive, you make cheating *financially suicidal* — a validator that misbehaves loses their staked ETH.

What is Proof of Stake

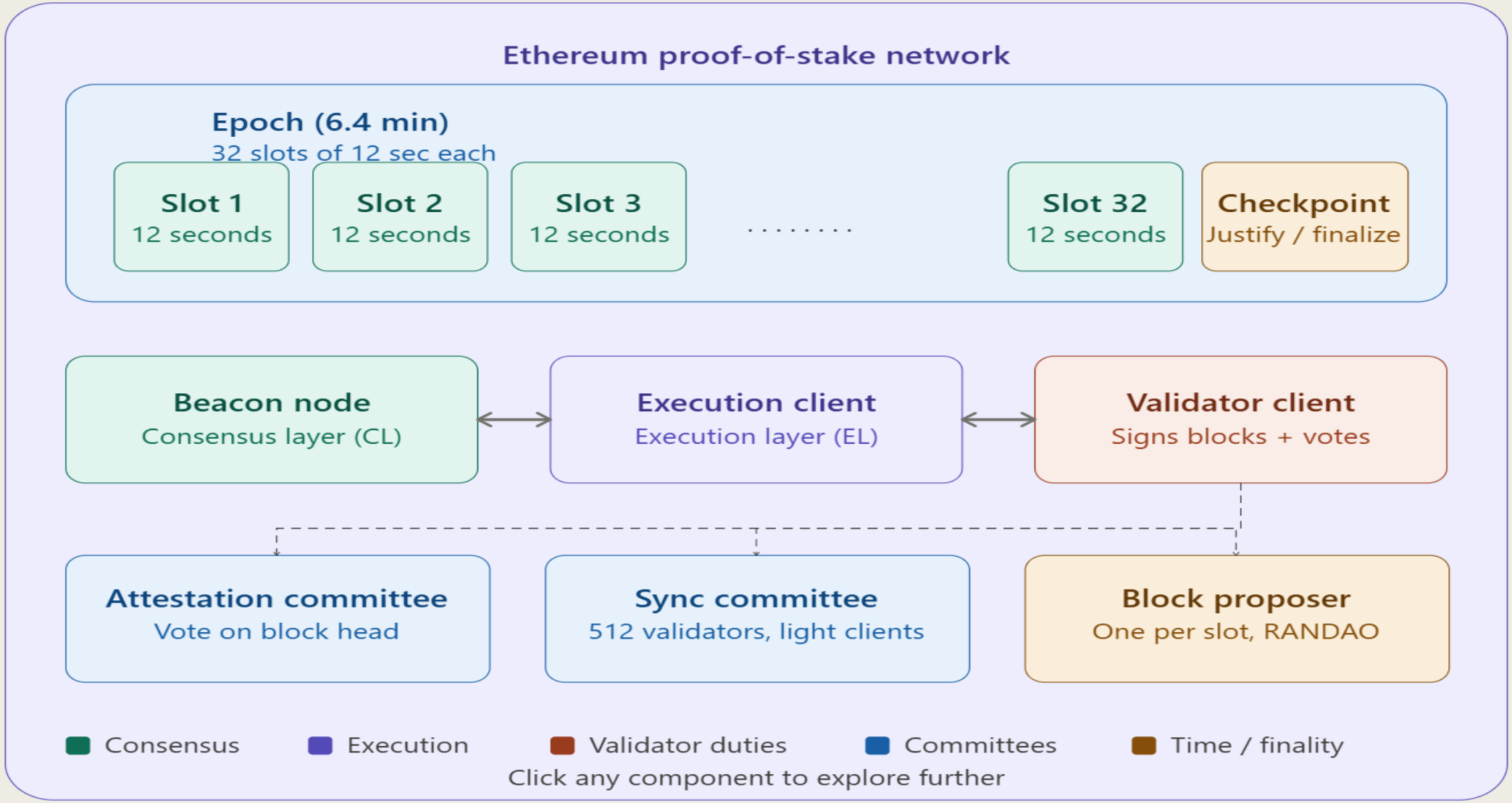
Proof-of-stake is a way to prove that validators have put something of value into the network that can be destroyed if they act dishonestly.

In Ethereum's proof-of-stake, validators explicitly stake capital in the form of ETH into a smart contract on Ethereum.

The validator is then responsible for checking that new blocks propagated over the network are valid and occasionally creating and propagating new blocks themselves.

If they try to defraud the network (for example by proposing multiple blocks when they ought to send one or sending conflicting attestations), some or all of their staked ETH can be destroyed.

The Process


Ethereum proof-of-stake network
Epoch (6.4 min)
32 slots of 12 sec each
Slot 1
12 seconds
Slot 2
12 seconds
Slot 3
12 seconds
Slot 32
12 seconds
Checkpoint
Justify / finalize
Beacon node
Consensus layer (CL)
Execution client
Execution layer (EL)
Validator client
Signs blocks + votes
Attestation committee
Vote on block head
Sync committee
512 validators, light clients
Block proposer
One per slot, RANDAO
Consensus
Execution
Validator duties
Committees
Time / finality
Click any component to explore further

Types of Nodes involved in Validation

he Three Types of Nodes

Ethereum's post-Merge architecture separates concerns across three distinct software components that run together on a validator's machine:

1. The Beacon Node (Consensus Layer Client) This is the heart of PoS. It runs the consensus protocol (Gasper), maintains the beacon chain, tracks the validator registry, and orchestrates the RANDAO randomness. It implements the fork-choice rule (LMD-GHOST) and drives finality (Casper FFG). Popular implementations: Prysm, Lighthouse, Teku, Nimbus, Lodestar.

2. The Execution Client (Execution Layer Client) This carries over from the PoW era. It runs the EVM, maintains the world state , validates transactions, builds execution payloads, and manages the mempool. It talks to the beacon node via the **Engine API** (a local RPC interface over HTTP). Popular clients: Geth, Nethermind, Besu, Erigon.

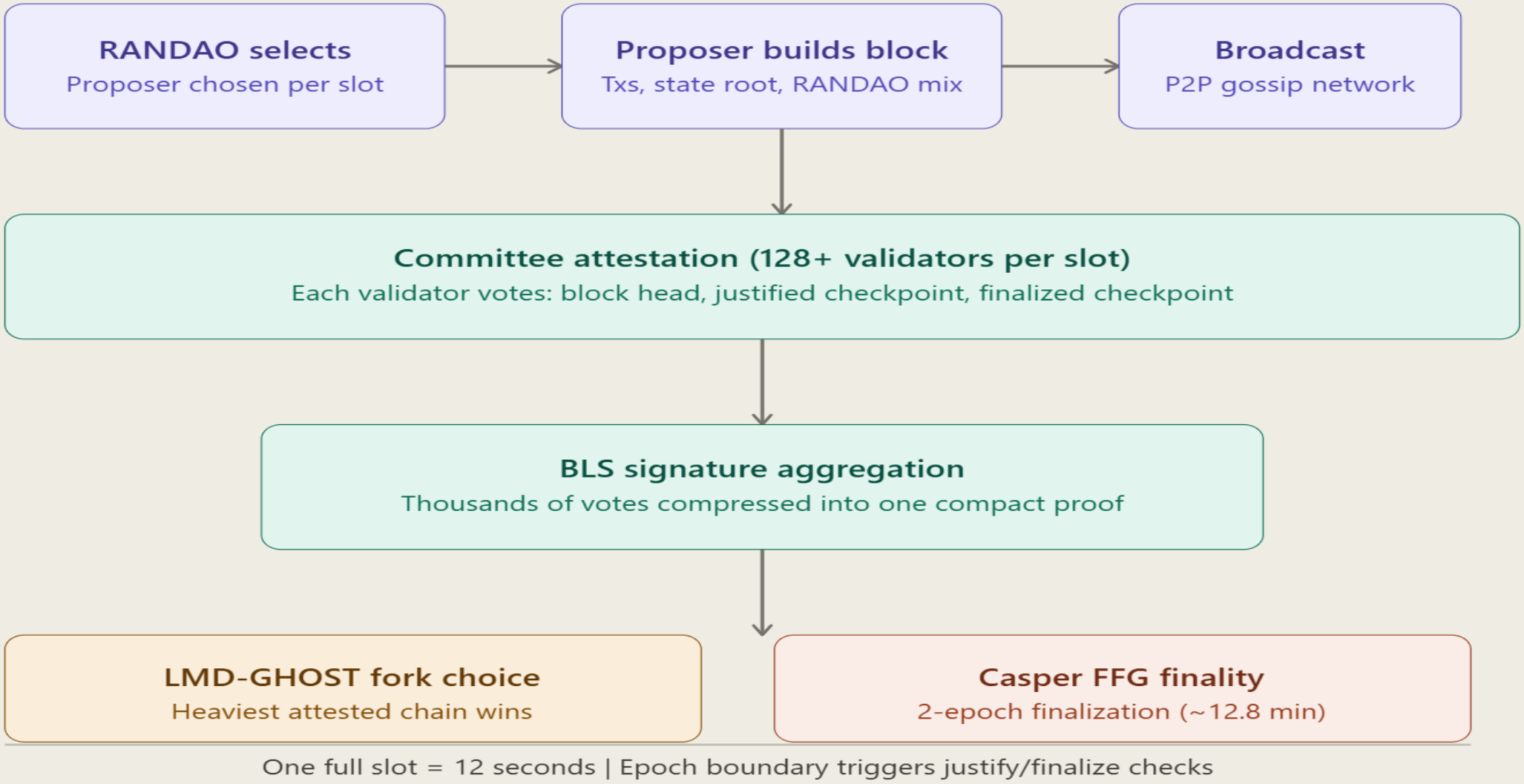
3. The Validator Client A lightweight process that holds validator signing keys and interfaces with the beacon node. It has two primary duties: proposing blocks when selected, and casting attestation votes every slot. It must never sign two conflicting messages (double-voting), or it faces slashing.

The Consensus Protocol

The Consensus Protocol: Gasper = LMD-GHOST + Casper FFG

- Ethereum's consensus is actually a *composition* of two sub-protocols working together. This is subtle and important:
- Now let us walk through the actual block proposal and attestation flow step by step:

The Consensus Protocol



Consensus Protocols

- **LMD-GHOST (Latest Message Driven Greediest Heaviest Observed SubTree)**
- This is the fork-choice rule — it answers: "given multiple competing chain tips, which one do I build on?" The algorithm traverses the block tree from the genesis block, and at each fork, picks the branch that has the *most recent attestation weight* behind it. Critically, only each validator's *latest* message counts (LMD — Latest Message Driven), which prevents validators from casting old votes to re-org the chain.
- **Casper FFG (Friendly Finality Gadget)**
- This is the finality layer, designed by Vitalik Buterin and Virgil Griffith. It overlays a voting mechanism on top of LMD-GHOST. Every epoch (32 slots = ~6.4 minutes), validators cast source → target votes linking pairs of *checkpoints* (the first block of each epoch). When ⅔ of the validator set votes for the same source → target pair, the source is *justified*. When two consecutive epochs are both justified, the earlier one becomes *finalized* — it can never be reverted without burning at least ⅓ of all staked ETH.
- This is the critical safety guarantee: reverting a finalized block requires **destroying at least ⅓ of the total stake** (~millions of ETH), which makes attacks economically infeasible.

Proof of Stake Validators

- To participate as a validator, a user must deposit 32 ETH into the deposit contract and run three separate pieces of software: an execution client, a consensus client, and a validator client.
- On depositing their ETH, the user joins an activation queue that limits the rate of new validators joining the network.
- Once activated, validators receive new blocks from peers on the Ethereum network. The transactions delivered in the block are re-executed to check that the proposed changes to Ethereum's state are valid, and the block signature is checked.
- The validator then sends a vote (called an attestation) in favor of that block across the network.

Slots & Epochs

- Whereas under proof-of-work, the timing of blocks is determined by the mining difficulty, in proof-of-stake, the time is fixed.
- Time in proof-of-stake Ethereum is divided into slots (12 seconds) and epochs (32 slots).
- One validator is randomly selected to be a block proposer in every slot.
- This validator is responsible for creating a new block and sending it out to other nodes on the network.
- Also in every slot, a committee of validators is randomly chosen, whose votes are used to determine the validity of the block being proposed.
- Dividing the validator set up into committees is important for keeping the network load manageable.
- Committees divide up the validator set so that every active validator attests in every epoch, but not in every slot.

How Transactions are Executed

The following provides an end-to-end explanation of how a transaction gets executed in Ethereum proof-of-stake.

- A user creates and signs a transaction with their private key. This is usually handled by a wallet or a library such as ethers.js, web3js, web3py etc but under the hood the user is making a request to a node using the Ethereum JSON-RPC API.
- The user defines the amount of gas that they are prepared to pay as a tip to a validator to encourage them to include the transaction in a block. The tips get paid to the validator while the base fee gets burned.
- The transaction is submitted to an Ethereum execution client which verifies its validity. This means ensuring that the sender has enough ETH to fulfill the transaction and they have signed it with the correct key.

Transaction Validation

- If the transaction is valid, the execution client adds it to its local mempool (list of pending transactions) and also broadcasts it to other nodes over the execution layer gossip network.
- When other nodes hear about the transaction they add it to their local mempool too.
- Advanced users might refrain from broadcasting their transaction and instead forward it to specialized block builders such as Flashbots Auction. This allows them to organize the transactions in upcoming blocks for maximum profit (MEV).

Block Proposer

- One of the validator nodes on the network is the block proposer for the current slot, having previously been selected pseudo-randomly using RANDAO.
- This node is responsible for building and broadcasting the next block to be added to the Ethereum blockchain and updating the global state.
- The node is made up of three parts: an execution client, a consensus client and a validator client.
- The execution client bundles transactions from the local mempool into an "execution payload" and executes them locally to generate a state change.
- This information is passed to the consensus client where the execution payload is wrapped as part of a "beacon block" that also contains information about rewards, penalties, slashings, attestations etc. that enable the network to agree on the sequence of blocks at the head of the chain.
- The communication between the execution and consensus clients is described in more detail in Connecting the Consensus and Execution Clients.

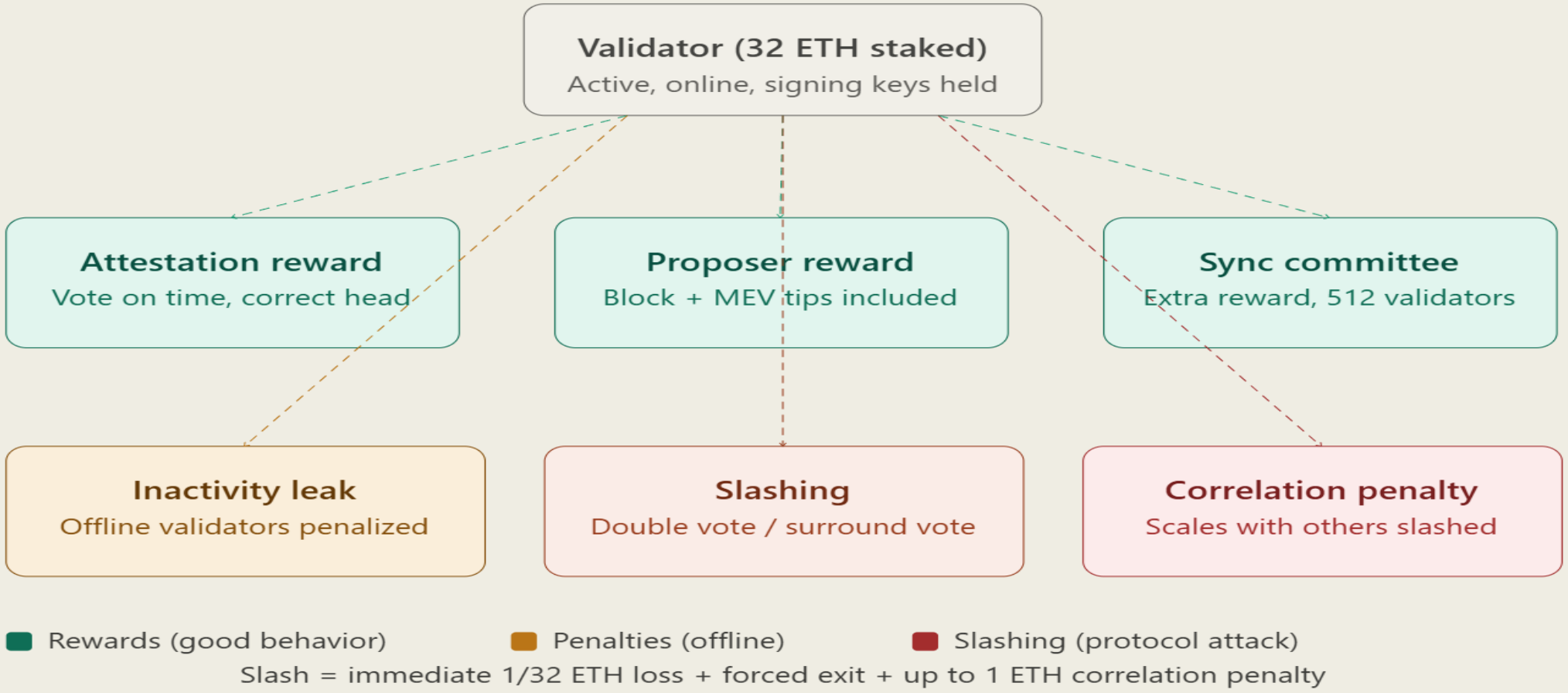
Validators Committee

- Other nodes receive the new beacon block on the consensus layer gossip network.
- They pass it to their execution client where the transactions are re-executed locally to ensure the proposed state change is valid.
- The validator client then attests that the block is valid and is the logical next block in their view of the chain (meaning it builds on the chain with the greatest weight of attestations as defined in the fork choice rules).
- The block is added to the local database in each node that attests to it.
- The transaction can be considered "finalized" if it has become part of a chain with a "supermajority link" between two checkpoints.
- Checkpoints occur at the start of each epoch and they exist to account for the fact that only a subset of active validators attest in each slot, but all active validators attest across each epoch.
- Therefore, it is only between epochs that a 'supermajority link' can be demonstrated (this is where 66% of the total staked ETH on the network agrees on two checkpoints).

Finality

- A transaction has "finality" in distributed networks when it is part of a block that can't change without a large amount of ETH getting burned.
- On proof-of-stake Ethereum, this is managed using "checkpoint" blocks.
- The first block in each epoch is a checkpoint. Validators vote for pairs of checkpoints that it considers to be valid. If a pair of checkpoints attracts votes representing at least two-thirds of the total staked ETH, the checkpoints are upgraded. The more recent of the two (target) becomes "justified". The earlier of the two is already justified because it was the "target" in the previous epoch. Now it is upgraded to "finalized". This process of upgrading the checkpoints is handled by Casper the Friendly Finality Gadget (Casper-FFG). Casper-FFG is a block finality tool for consensus. Once a block is finalized, it cannot be reverted or changed without a majority slashing of stakers, making it economically inviable.

The Economics: awards mad Penalties



Slashable Offenses

Slashable Offences (The Hard Rules)

There are exactly two categories of slashable behaviour:

- **Equivocation (Double Voting):** A validator signs two different blocks for the same slot. This is the proposer version of the offense.
- **Surround Voting:** A validator casts two Casper FFG votes where one "surrounds" the other (i.e., a.source < b.source and b.target < a.target). This is the most dangerous attack because it can be used to violate finality.
- Both are detected automatically by other validators who submit *slashing proofs* to the chain and receive a portion of the slashed validator's stake as a reward for doing so.
- The **correlation penalty** is a particularly elegant design: if you're slashed at the same time as many other validators (which suggests a coordinated attack or a shared infrastructure failure), your penalty scales up to a maximum of 100% of your stake. A lone misconfigured validator loses roughly 1 ETH; an attacker coordinating 33% of validators loses everything.

The RANDAO: Decentralized Randomness

he RANDAO: Decentralized Randomness

- Block proposer and committee selection must be random and unpredictable. Ethereum uses **RANDAO** — a commit-reveal scheme:
- Every block proposer mixes their BLS signature (a hash of the current epoch) into a running "RANDAO accumulator." The result is the seed for the shuffling algorithm (SWAP-OR-NOT) that assigns validators to committees and slots for the next epoch.
- Because each proposer adds entropy, a single malicious proposer can only bias the randomness by choosing *not to propose* (which costs them their block reward). Biasing by more than 1 bit per epoch requires controlling many consecutive proposers — extremely unlikely and very expensive.
- For applications requiring stronger randomness guarantees, Ethereum is migrating toward **VDF (Verifiable Delay Functions)** in a future upgrade, which makes lookahead attacks impossible by design.